Cyberboswachters

De beste digitale stropers zijn ook de beste cyberboswachters

Dams Tim

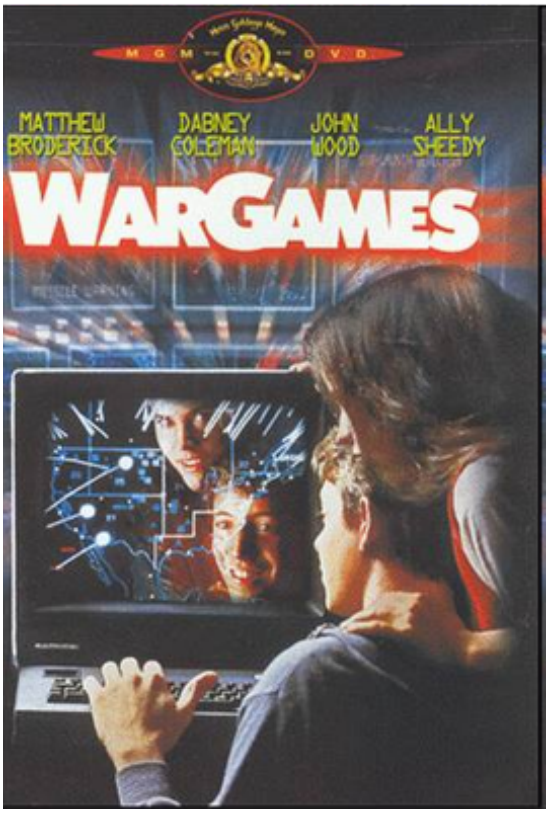
Wifi security

Alhoewel dit hoofdstuk integraal na het crypto hoofdstuk komt, is het toch interessant om dit hoofdstuk geschrankt met het crypto hoofdstuk door te nemen als volgt: * “Crypto” tot en met “Symmetric stream ciphers”. * “Wifi security” tot en met “WEP en waarom het faalde”. * De rest van “Crypto”. * De rest van “Wifi security”

De problemen van Wifi

We kunnen draadloze netwerken, specifiek wifi-netwerken, niet meer uit ons leven inbeelden. De opkomst van de IEEE 802.11b standaard in 1999 veroorzaakte een kleine revolutie in de manier waarop bedrijven en privégebruikers konden werken. Plots kon je met een laptop van overal in het gebouw - en zelfs er buiten- op het netwerk geraken. Die vrijheid voor de gebruikers betekende wel een nachtmerrie voor de cyberboswachters. Een netwerkkabel heeft een intrinsieke extra beveiliging: enkel daar waar de kabel ligt kunnen gebruikers aan het netwerk geraken. Zolang je dus geen netwerkkabel bijvoorbeeld naar de publieke parking brengt kan niemand van daar illegaal het netwerk benaderen. Met wifi leek het alsof plotseling het hele netwerk in een straal van tientallen meters rond het gebouw beschikbaar was, met alle gevolgen van dien.

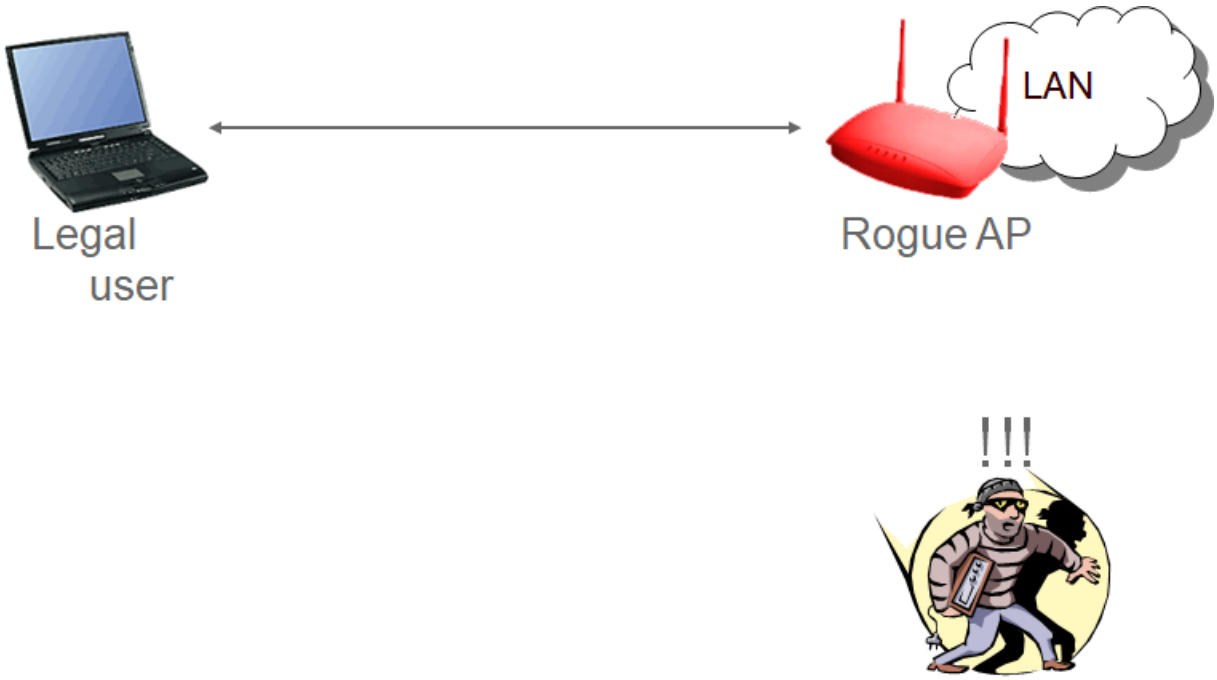
Al gauw werd een nieuwe sport uitgevonden door hobbyist hackers en professionele cybercriminelen: **wardriving**. De idee is eenvoudig: rijdt rond in de stad en laat de laptop naast je in de auto scannen naar alle netwerken, met extra aandacht voor die netwerken die geen of zwakke beveiliging hadden.



De term wardriving komt van de term *wardialing* die op zijn beurt gebaseerd is op de klassieke cyber-cult film “Wargames” uit 1983. Voor de geschiedkundigen onder ons, wardialing was het opbellen van willekeurige telefoonnummers met je modem in de hoop een zogenaamd *bulletin board system oftewel **BBS** (een pre-internet forum zeg maar) te vinden.

Draadloze netwerken die gevonden worden hebben dan ook een schare aan problemen:

- **Eavesdropping**: iedereen kan “meeluisteren” wat er door de lucht vliegt. Dit heb je niet met een bedrade kabel.
- **Invasion**: je kan eenvoudig verbinden met het netwerk indien er geen beveiliging voorzien werd. Iets dat je dus bij een bedraad netwerk enkel kunt als je fysiek toegang hebt tot een netwerkkabel.
- **Man-in-the-middle aanvallen**: hier gaan we zo meteen dieper op in.
- **Backdoor**: een veelvoorkomend probleem zijn zogenaamde **rogue access points** die worden bijgeplaatst op het netwerk door goedbedoelde werknemers die zou het het bereik van het netwerk wat willen uitbreiden. Vaak zijn echter de beveilingsinstellingen van zo’n (meestal goedkoop) access point niet zo sterk als die van het bedrijf. Bijgevolg is dit de ideale backdoor voor malafide gebruikers die het netwerk aan het surveileren zijn. Wanneer ze een netwerkscan doen zullen ze tientalle goed beveiligde access points zien en 1 zwak beveiligd.



Figuur 0.1: Een rogue access point is de ideale manier om binnen te geraken voor hacker.

- **Denian-of-service op fysiek niveau**: om een gebruiker toegang tot een bedraad netwerk te ontzeggen op fysiek niveau dien je de kabel door te knippen. Bij wifi is dit nog eenvoudiger: de "kabel" bij wifi zijn de frequentiebanden in de lucht waarbinnen de apparaten mogen werken (circa 2.4 Ghz bij de oudere wifi apparaten, nu meestal rond de 5 Ghz band). De wifi-apparaten kunnen enkel met elkaar communiceren indien zij een signaal naar elkaar over die frequentie-band kunnen sturen op een moment dat niemand anders in de buurt die band gebruikt (zie note hierna). Als een malafide gebruiker dus die frequentieband vult met andere signalen, dan zullen de legale gebruikers nooit iets kunnen uitsturen. Wil je dus een wifi netwerk *Dos'n*, koop dan een signaalgenerator die op de juiste frequentieband de nodige ruist uitzend en klaar is kees.

Bedrade netwerk apparaten werken volgens het CSMA/CD (Carrier Sense Multiple Access / Collision Detection) om met elkaar over de draad te communiceren, hierbij detecteren ze wanneer er 'botsingen' tussen pakketen voordoen en de informatie dus opnieuw moet uitgestuurd worden. Bij draadloze netwerken is het echter onmogelijk om *botsingen in de lucht* te detecteren, daarom werken ze met een variant: **CSMA/CA** , oftewel CSMA/ **Collision Avoidance**. Een wifi-apparaat dat iets wil uitsturen zal eerst controleren of de frequentieband waarop ze werken vrij is, enkel dan zal er vervolgens een pakketje de lucht worden ingestuurd.

Sommige gebruikers schakelen het broadcasten van hun netwerknaam (het zogenaamde **SSID**) uit in de hoop dat zo dat buren, voorbijgangers en wardrivers hun netwerk niet kunnen zien. Helaas is dit zogenaamde *snake's oil*: het geeft een vals gevoel van veiligheid. Je kan weliswaar SSID-broadcasting uitzetten (dit is een pakketje dat je access point elke paar seconden uitstuurt om aan iedereen die luistert te zeggen *"Hallo, hier is het netwerk met naam X, en dit zijn de parameters die je nodig hebt om met mij te verbinden"*) maar het SSID wordt ook in een hoop andere pakketjes de lucht in gestuurd. Een beetje wifi hacker kan dus het SSID ogenblikkelijk uit de lucht plukken, ongeacht dat broadcasting werd uitgeschakeld of niet.

De originele wifi beveiliging

Om de huidige, en betere, beveiliging van wifi te appreciëren gaan we wederom even terug in de tijd om te kijken hoe de originele IEEE wifi standaard de beveiliging beschreef. Het zal een ietwat horror-achtige tocht worden waarin we gaan ontdekken dat enkele stevige hiaten ervoor gezorgd hebben dat illegale toegang tot bijna ieder wifi netwerk rond de eeuwwisseling binnen enkele minuten kon gebeuren. Lees verder en huiver mee.

Onbeveiligde managementframes

Veel keuzes die in de IEEE standaard werden gemaakt zijn vermoedelijk het gevolg dat men leentje buur is gaan spelen bij de reeds bestaande IEEE standaarden voor bedrade netwerken. Echter, bij bedrade netwerken had je niet de inherente onveilige omgeving van het draadloze aspect, waardoor op gebied van beveiliging hier weinig tot geen aandacht aan werd besteed.

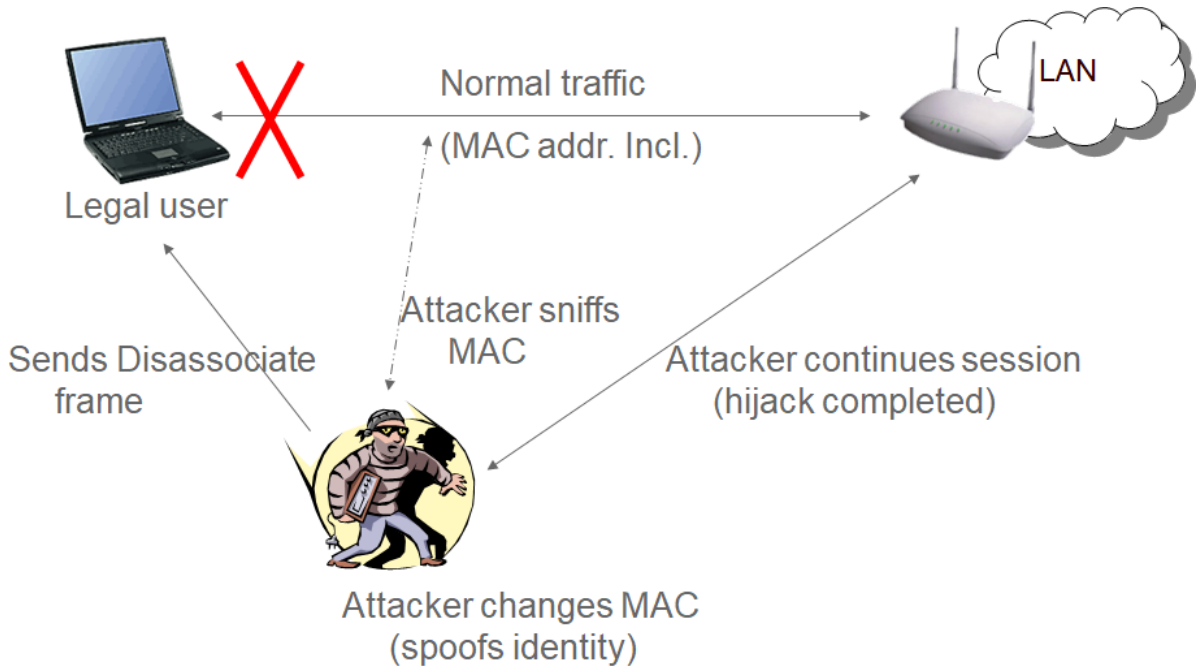
Management frames in wifi zijn frames die ervoor zorgen dat alle aanwezigen op het netwerk op een ordentelijke manier met elkaar kunnen communiceren. Deze frames zorgen bijvoorbeeld voor:

- Om een client verbinding te laten maken met een netwerk.
- Om SSIDs te broadcasten.
- Clients de opdracht te geven het netwerk te verlaten.
- Om te verbinden met een ander access point van hetzelfde netwerk (*handover*).

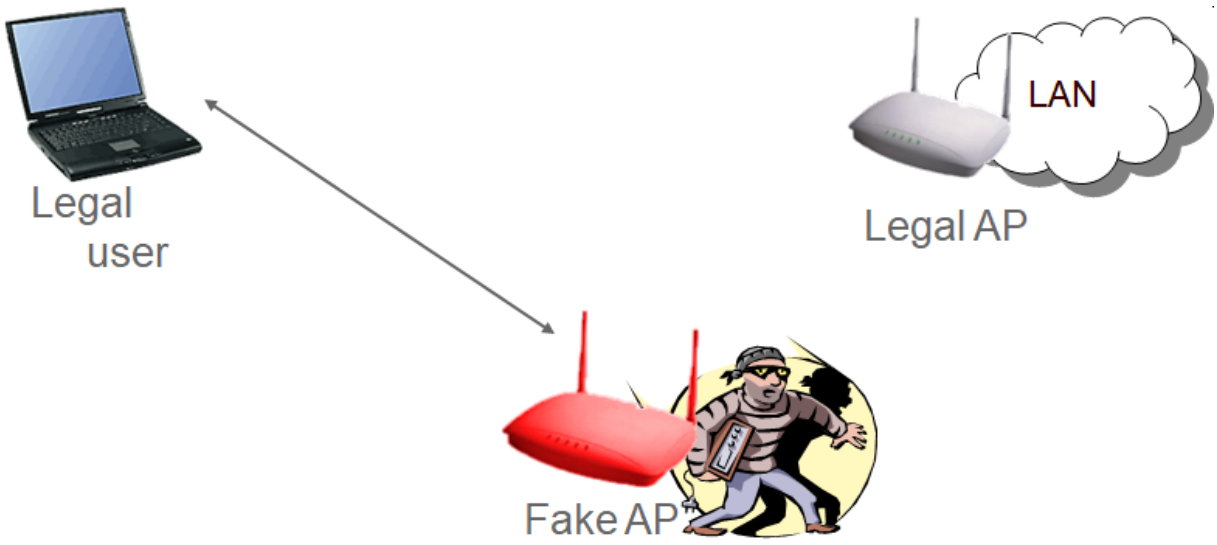
Net zoals bij bedrade netwerken, koos men bij de IEEE wifi standaard om deze managementframes **zonder enige vorm van beveiliging** te gebruiken (er werd geen *CIA* voorzien). Dit zorgde ervoor dat niet legale gebruikers zelfs management frames konden uitsturen op een netwerk, zonder dat de ontvangers ervan konden controleren of de bron wel een legaal access point of client was.

Dit resulteerde in onder andere volgende scenario's:

- Een hacker kan legale gebruikers DoS'n door constant zogenaamde *disassociation* naar hen te sturen. Dit frame, gebruikt door access points, geeft aan clients de opdracht dat ze het netwerk moeten verlaten. De hacker kan zo'n frame uitsturen en daarbij het "source" veld instellen op het MAC-adres van het access points. Dit spoofing kan ongecontroleerd, waardoor legale gebruikers dit frame altijd zullen aanvaarden én vervolgens uitvoeren: de gebruiker kan niet meer verbinden met het netwerk zolang de disassociation frames blijven verstuurd worden (** disassociation flooding**)
- **Identity spoofing** was ook eenvoudig daar de hacker eender welk veld in de frames kan aanpassen. Van zodra hij een legale gebruiker met voorgaande techniek van het netwerk heeft geschopt, kan hij vervolgens zichzelf voordoen als deze gebruiker. Hiervoor moet hij gewoon het MAC-adres spoofing van de legale gebruikers tijdens de communicatie met het access point.



- En last but not least laten de onbeveiligde management frames toe dat we eenvoudig een access point kunt nabootsen (**impersonation**). Vervolgens kunnen we een **man-in-the-middle aanval** uitvoeren daar een hacker zich kan plaatsen tussen de gebruiker en het internet en zo informatie kan ontfutselen.



De linux tool *AirSnarf* laat toe om fake hotspots (publiek wifi netwerk) op te zetten. Hierbij maakt het gebruikt van de onbeveiligde management frames. Een scenario in België dat gegarandeerd success heeft (vanuit het standpunt van de hacker) is een fake Telenet Wifree hotspot op te zetten. Hierbij zal je eerst de login pagina van Telenet Wifree kopieren en via een lokale webserver aanbieden aan de gebruikers die op jouw access point met de naam “Telenet Wi-Free” verbinden. Je kan nu van iedere gebruiker de gebruikersnaam en paswoord stelen telkens deze die informatie op jouw fake pagina invoert.

Volgende Engelstalige teksten koment uit een oudere cursus van dme en zullen (ooit) vertaald worden. Op deze manier kan ik echter focussen op jullie zoveel mogelijk leerstof in cursusvorm aan te bieden.

Phase 1.1 : WEP, the original security

The original “ANSI/IEEE Std. 802.11” was written in 1999 and had the purpose “to develop a medium access control (MAC) and physical layer (PHY) specification for wireless connectivity for fixed, portable, and moving station within a local area.”
The security chapter in the standard (Chapter 8: “Authentication and Privacy”) was only a mere 10pages long, compared to the total number of pages (528) this might be seen as a forebode of how little security was originally conceived in the standard.

Before we dive into WEP, the original *security motor* of wifi, we will first have a look how clients actually join a wireless network. As previously noted, this prcoes includes the usage of unprotected management frames.

Joining a network

The moment there is any form of interaction between the attacker and his target he can begin his malicious acts. The same applies to wireless hacking: before being to hack a network the attacker first needs to find one and establish a 'link', in this case this can be nothing more but an antenna that captures all passing radio signals. Yet, capturing data passively as described is one thing, actively attacking a network is a whole other business. To be able to do so, an attacker needs to actually join a network, one way or another.

The IEEE 802.11 standard specifies how a wireless LAN can be joined. Several steps need to be undertaken before a user or attacker can be granted actual permission to the network and use it for higher-layer data traffic: 1. **Scanning**: Searching for possible networks to join. 2. **Joining**: Choosing a network you want to access. 3. **Authentication**: Giving the right credentials to be allowed to use the network. 4. **Association**: Making sure the system keeps track of your location in the network. It is important to note that a user or attacker can only begin using a network's resources when he is associated.

Scanning Any device that wants to use a network first needs to find one, and so it scans the area to discover a compatible network to join. Several parameters are used in the scanning procedure and some of them can be specified by user; many implementations have default values for these parameters in the driver. One of these parameters is the SSID, or Service Set Identifier, which contains the name of the network. With this parameter, the user can choose to scan for a specific network, or scan for any network in the area using the broadcast SSID.

The SSID is a 32byte ASCII character string as described in the 802.11 specifications. In these specifications is also stated that any client setting this string to a 'NULL' string will associate to any access point regardless of the SSID setting on the access points. This is referred to as the broadcast SSID and is normally only used in Probe Request frames when a station attempts to discover all the 802.11 networks in its area.

Joining After the scan report is made the station can choose to join one of the networks. This joining is not the same as associating; it is analogous to aiming a weapon, you are only planning to use the chosen network and its services, but first you have to be authenticated one way or the other and be associated.

What network is joined depends on whether the user chooses one or not. If the user lets the device choose, the station works with some criteria to make the decision like the power level and signal strength.** When chosen, the station has to synchronize its timing and also match the physical and MAC parameters.** It then tries to authenticate itself.

Authentication When in a wired network, authentication is almost provided by the mere physical access; if you are close enough to plug in a cable, you most likely were allowed by, for example, the receptionist at the front desk and thus need no extra authentication to use the network.

This is certainly not the fact in a wireless LAN. Therefore authentication was added to the process of joining a network nonetheless. Two major approaches are specified by 802.11 to ensure that a station attempting to associate with a network is allowed to do so:

- **Open-system authentication** (might include WEP)
- **Shared-key authentication** (includes WEP)

802.11 authentication as originally specified as a one-way street. There is no mutual authentication whatsoever, thus allowing a malicious attacker to employ 'man-in-the-middle' attacks (see earlier) without the end user knowing it. A rogue access point could send Beacon frames for a network it is not part of and for example attempt to steal authentication credentials.

No other authentication algorithms are defined in the 802.11 standard but the two mentioned before; there exists however a third popular (but not yet standardized) method,

- **MAC Address Authentication using a MAC-ACL**: this is simply a whitelist (ACL stands for **access control list**) containing all the MAC-addresses that are allowed on the network. However, as we've seen, this is useless snce MAC spoofing in wifi is peanuts (including sniffing the air for a legal MAC-address).

Open-system authentication

Originally the open-system authentication was seen as the 'no-security' method of authentication for wireless networks. History however has proven that this mode is now more secure when used in 802.1X since it does not leak any information on the used encryption etc (more about this later). Two frames are exchanged in this setup:

1. From client to access point, requesting access .
2. The other way around, with the access point basically sending an "hello there" frame.

If no encryption is used in the network, any device knowing the SSID of the AP can gain access to the network. With WEP encryption enabled, the WEP key itself becomes a form of access control. If a device does not have the correct WEP key, even though authentication was successful, the device will be unable to transmit data through the AP. And neither can the device decrypt data it receives from the AP. This way however, there is no security against attackers that have stolen/cracked the WEP key; there is no form of real user authentication.

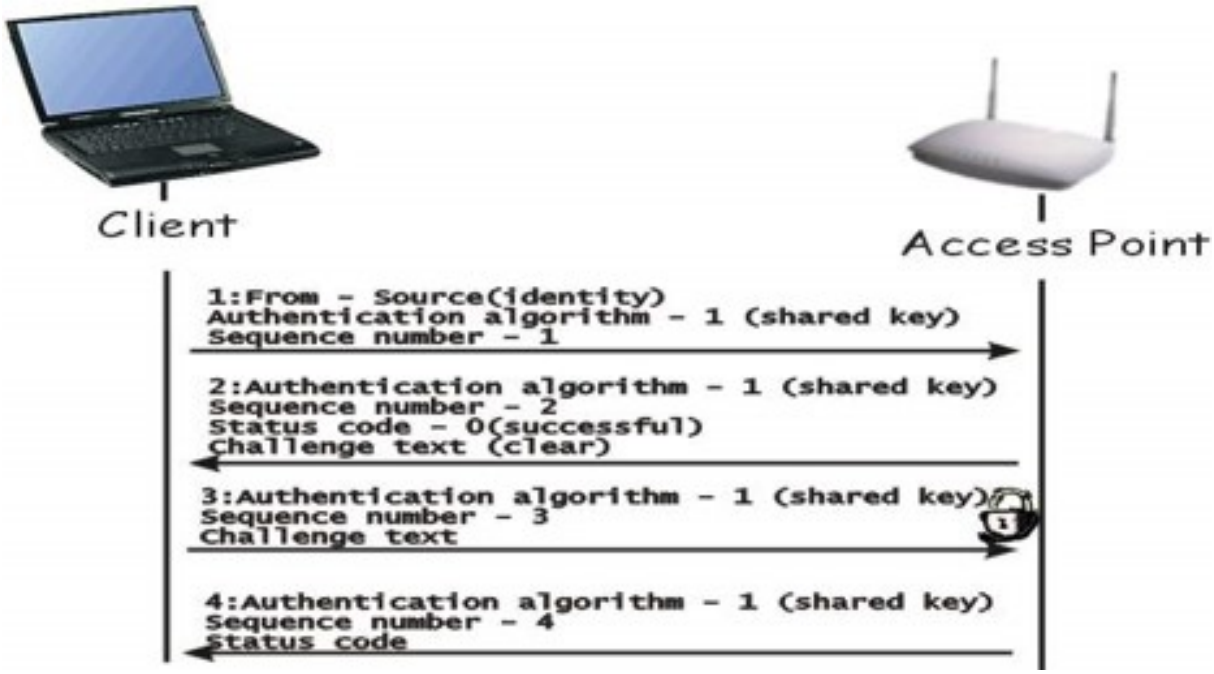
::: note It may seem useless to have an authentication algorithm like this that provides no real security when no other encryption is used. However, there are many wireless devices that simply don't have

enough CPU resources to support more complex authentication algorithms. Hand-held devices like bar-code reader, network scanners etc just need quick access to a network without the extra hassle of authentication and therefore could use open-system authentication.

Shared-key authentication

Shared-key authentication, as its name implies, requires that a shared key is distributed to the stations before they attempt to authenticate. This is done using WEP and therefore can only be used with products that implement WEP. Proving that you own the correct WEP key is enough proof to be allowed on the network.

This proof is accomplesh through a **challenge-response** system: 1. The access point creates a random string (*the challenge*) and sends this in plaintext to the client 2. The client encrypts this string and sends it back to the access point (*the reponse*) 3. The access point will try to decrypt the response using his own WEP key: if the original challenge text appears, the access point knows that the response was encrypted with the correct key and so will grant access to the client on the network.



Association Once authentication is completed, the station is allowed to associate and/or reassociate with any access points in the network, for which he is authenticated.

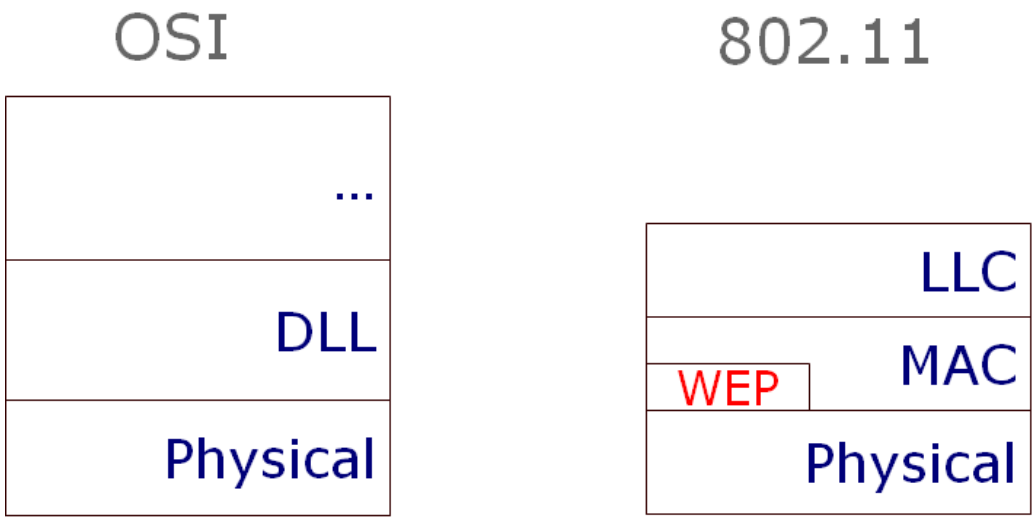
Once associated, actual data transmissions (i.e. usage of the network) can start. Depending on the settings of the network, it is at this point that actual encryption of the data starts. This encryption is accomplished using WEP, as explained next.

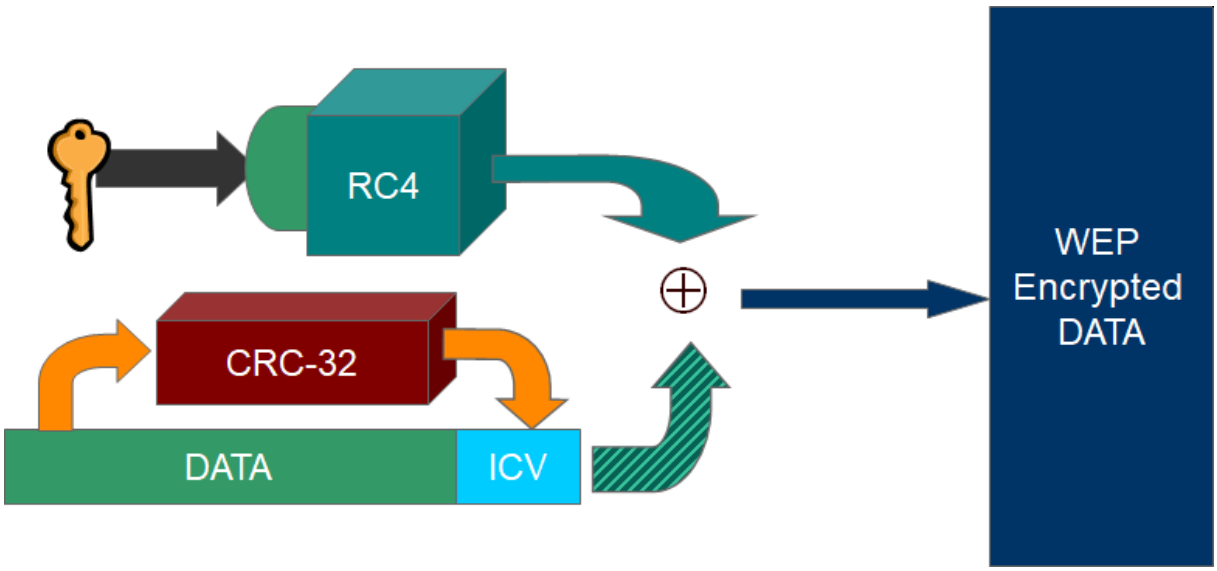
WEP

After being associated to a wireless network, the eavesdropping problem (and others) become obvious. Therefore the 802.11 specifications described **WEP** or **"Wired Equivalent Privacy**", a protocol that was believed to create the same level of privacy experienced on a wired LAN. WEP is 802.11's optional encryption standard implemented in the MAC Layer that most wireless LAN-cards and access point vendors supported around the time Wifi become immensely poppular.

When WEP is enabled, all data is encrypted before being transmitted. Only with the right key can the receiver decrypt the data afterwards. And so, when a user is finally associated with a network, he can rely on WEP to have some form of privacy.

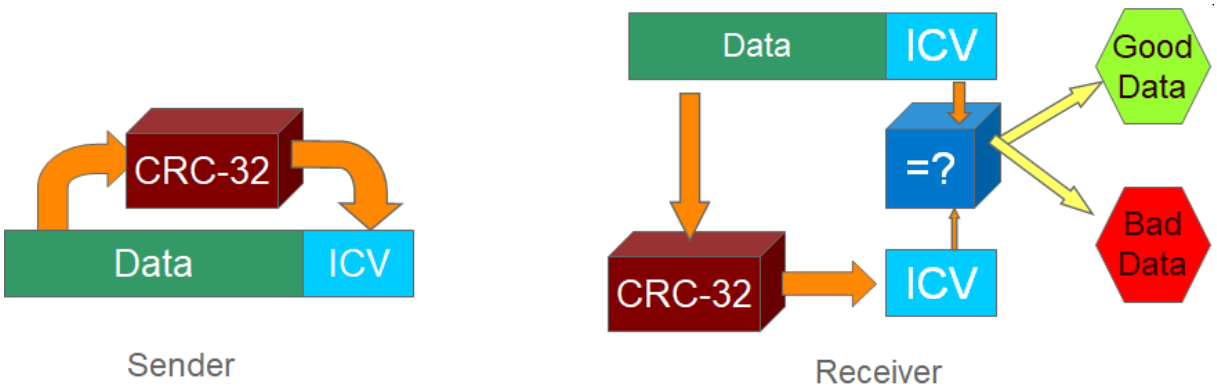
To be exact: the MAC layer receives packets from the LLC layer. If WEP is enabled this packet is sent to WEP where it is fragmented, if needed, into several frames. It is these frames that are encrypted and sent further down, to the PHY layer.z





Figuur 0.2: WEP in full

How WEP works WEP uses RC4, a symmetric stream cipher and a WEP-key to create a pseudo random key stream. It is based on the principle of a one-time pad, which is the only known encryption scheme that is mathematically proven to protect against certain types of attacks. The RC4 algorithm is a set of rules used to expand the key into a key stream as a explained earlier. This generated stream is XOR'd with the plain text producing the cipher text. To read this text again, the receiver needs to have the same key. Using this key he then recreates the same pseudorandom stream and XOR's the cipher text.



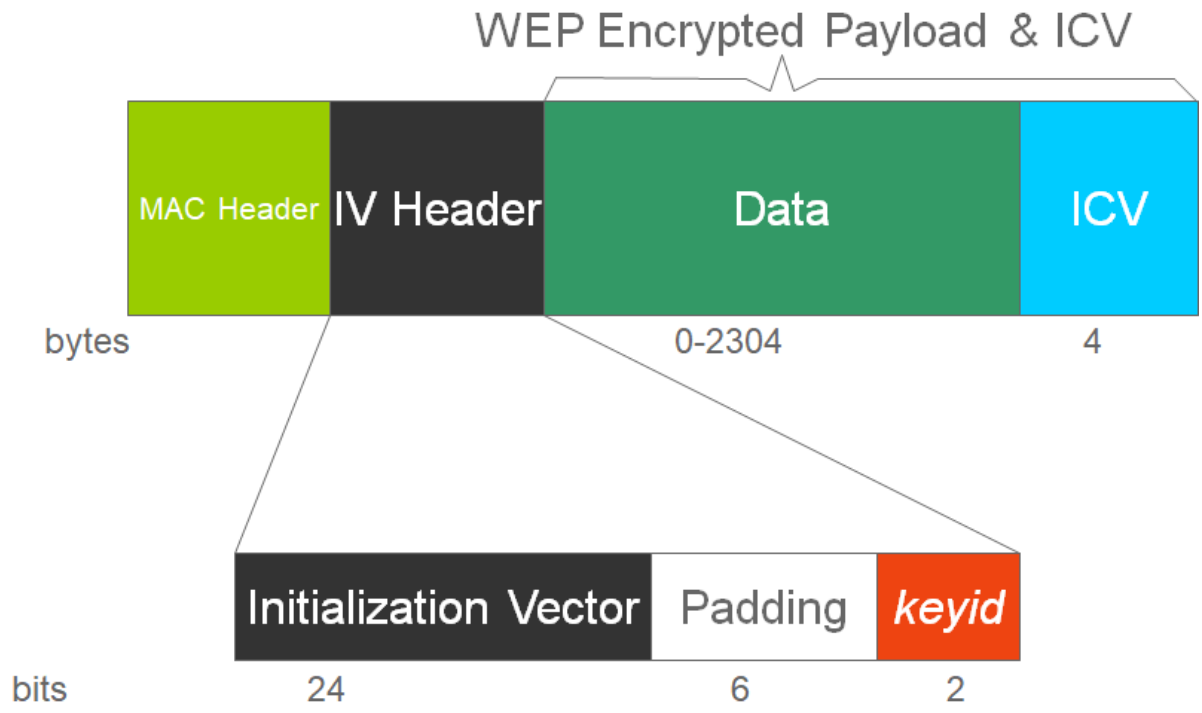
Figuur 0.3: Integrity check

Confidentiality and integrity are handled simultaneously. Before the encryption, the frame is run through an integrity check algorithm (CRC-32), generating a hash called the integrity check value (ICV).

This ICV protects the data from being forged during transmission. Both the frame and the ICV are encrypted making the ICV unavailable to casual attackers.

The decapsulation (decryption) is basically the reverse procedure, with that respect that the receiver also makes a CRC-32 hash which is compared to the one received. If both received and self-made hash are equal the data is considered 'clean' (i.e. was not changed due to random noise, etc)

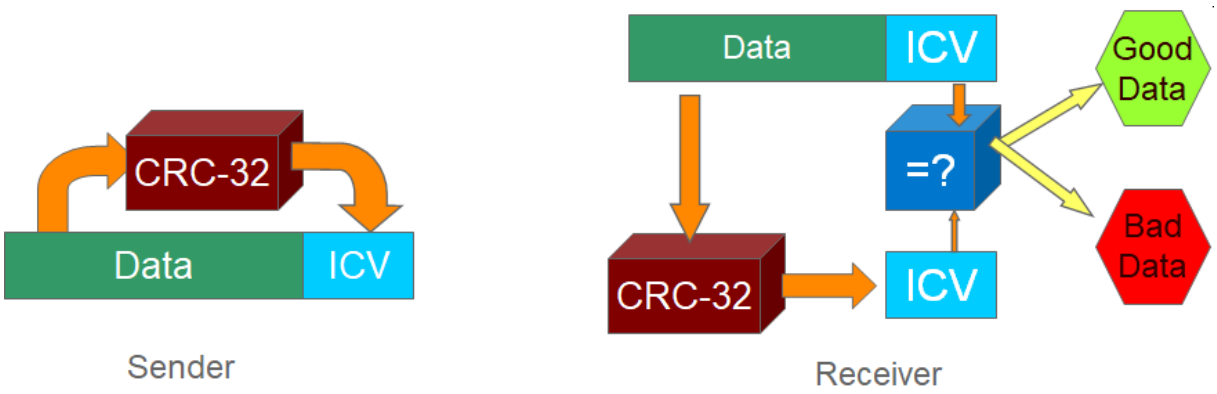
Shown in the following figure is a schematic overview of the resulting frame:



Figuur 0.4: WEP frame layout

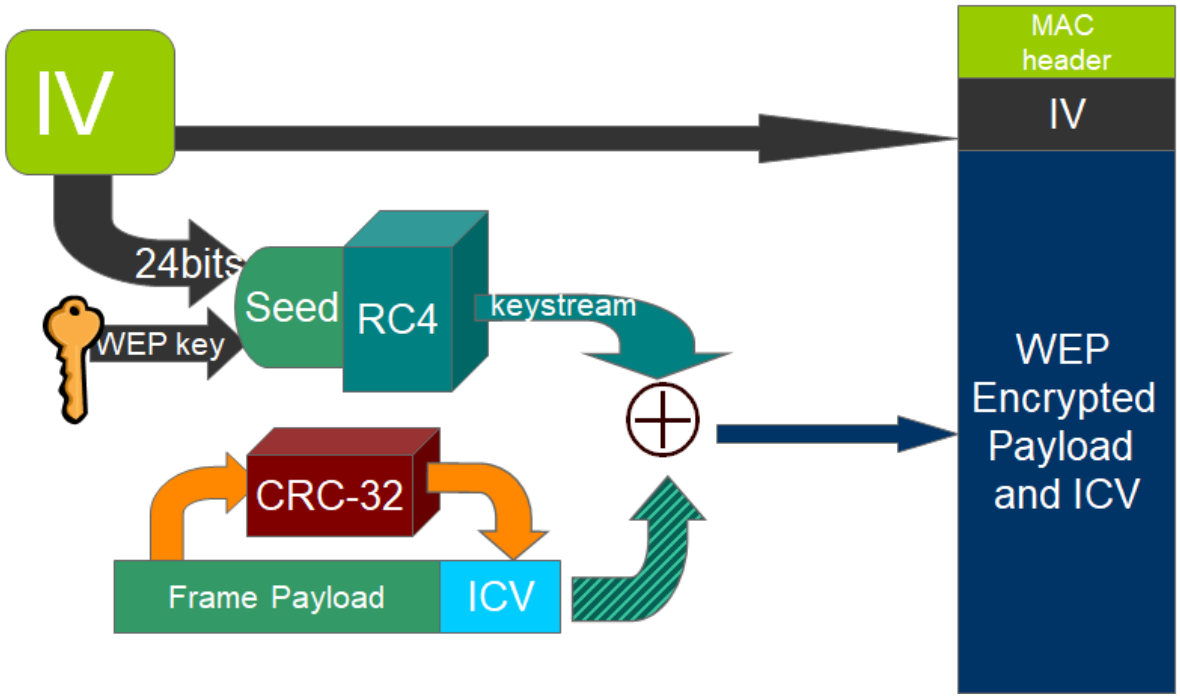
To protect traffic from brute-force decryption attacks, a set of up to four default keys is used. The default keys, identified by a variable keyid (0 to 3), need to be shared among all stations in a service set. Once a station has obtained the default keys for its service set, it can communicate using WEP.

WEP originally specified the use of a 40bit secret key; proprietary versions however also support longer keys (e.g. 104, 128, etc). The secret key is combined with a 24-bit initialisation vector (IV) to create a longer key (64 if 40-bit key was used). The IV is a random generated number; however this was not stated in the original specifications. This new key (WEP-key + IV) is used as the seed for the RC4 key stream generator. The key stream is then XOR'd with the frame body and the ICV.

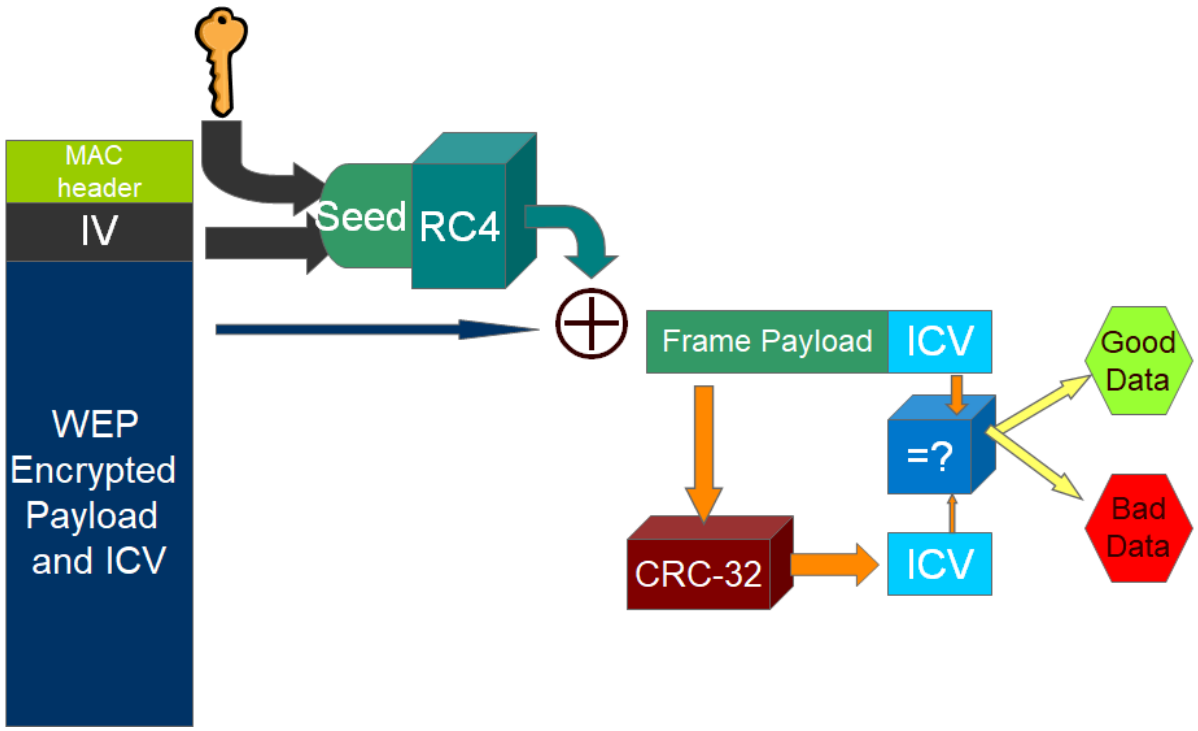


Figuur 0.5: Encryption

To enable the receiver to generate the same random stream, thus enabling him to decrypt the frame, the IV and keyid is placed in the header of the frame.



Figuur 0.6: WEP encryption in full



Figuur 0.7: WEP decryption in full

Phase 1.2: How WEP failed

Throughout time, more and more papers were released, identifying flaws in the overall WEP security.

The flaws can be summarized to 4 large problems with WEP, each having their own problematic results:

- RC4 cipher was not meant for datagram environments.
- The IV is badly implemented.
- The CRC-32 is not secure enough.
- No real key distribution system.
- There is no replay protection.
- The problem of having no replay protection will not be discussed separable since it is basically a flaw that 'helps' to make the four other mentioned problems even bigger.

Problem 1: RC4

Most problems result from a misuse of the RC4 cipher. It is used in a large range of modern security devices, because it provides a high degree of privacy and that for a relatively low performance penalty

(consumes very little power since it uses no multiplications).

However, stream ciphers in general, RC4 in particular, are questionable choices in an unreliable datagram environment like that of WEP. In a datagram environment, the same packet is often resent because of a transmission error, which happens as much as 20% of the transmissions. This is normal, but unwanted for a secure stream cipher. Because of this datagram environment, two properties of a stream cipher produce severe privacy gaps that, in conjunction with the IV and other flaws, can be abused by attackers.

RC4 has no random access property Before we actually begin discussing the security related flaws, it should be noted on why RC4 actually was a bad choice by the IEEE 802.11 group when it comes to encryption and decryption speed on the MAC-layer.

The loss of a single bit of a data stream encrypted under RC4 causes the loss of all the data following the lost bit. This is because a data loss desynchronizes the RC4 encryption and decryption engines. A full reset of both the engines is then the only real solution.

Since IEEE 802.11 MAC is neither reliable nor delivers-in order at the level of which WEP operates, WEP requires that the cipher support a random access, "seek" type capability, where it is wanted to instantly and efficiently switch the cipher to any selected point in the key stream and not having to begin from the start after an error.

Instead of selecting a stream cipher with characteristics needed for a datagram environment, the WEP architecture tries to accommodate itself by reinitializing the cipher key schedule on every data frame. Creating an overhead that could have been avoided when a cipher with random access was chosen (one such as AES).

RC4 disallows key re-use Stream ciphers have a second property that is important: it is unsafe to use the same key twice, ever.

Presume you have two plaintext byte sequences $p_1, p_2, p_3, \ldots$ and $q_1, q_2, q_3, \ldots$ both which you encrypt with the a key stream $k_1, k_2, k_3, \ldots$.This encryption, as we have seen, is an XOR of the key and plain text. So we have two new cipher texts:

$p_1 \oplus k_1, p_2 \oplus k_2, p_3 \oplus k_3$

$q_1 \oplus k_1, q_2 \oplus k_2, q_3 \oplus k_3$

Now, presume an attacker captures these two cipher texts; what then follows is a failure of privacy, since:

$(p_i \oplus k_i) \oplus (q_i \oplus k_i) = p_i \oplus q_i$

Or in other words, combining two cipher texts produces a stream, which is not dependant of the used key, and so a large deal of information about the plain texts is revealed. If one of the two plain texts is known, the other can be read, if it has the same length, without the need for a key, a property that will be abused in the following flaws.

As a result, stream ciphers are insecure in a datagram environment without some sort of key management to replace keys before they can be reused. The WEP design attempts to accommodate this lack of key management by introducing the IV. WEP combines the IV with the key to produce a new frame specific encryption key, preventing that any collisions of two or more frames with the same key occur (explained further on).

Problem 2: IV

The WEP designers had some knowledge of the first flaw concerning RC4 and therefore introduced a per-packet key (the IV). From a cryptographic view, this is a good solution.

However a major flaw is the fact that no replay protection is implemented whatsoever which shall soon be explained.

IV gives rise to weak keys The paper by Scott Fluhrer, Itsik Mantin, and Adi Shamir presented in August 2001 investigated the RC4 key schedule when a portion of the RC4 key stream is known. Explaining the paper and its consequences in depth requires some advanced mathematical knowledge, thus only a summary is provided:

The paper showed when a part of the RC4 key stream is known, a class of RC4 weak keys could be identified. When these weak keys are used to generate a pseudo random stream there is a small, but not insignificant, correlation between the input (the WEP key) and the output (the key stream).

In other words: weak IV are the cause of some key leakage to the key stream which, of course, is a very unwanted property of any type of cryptographic cipher.

As a result of these so-called weak keys, if the first two bytes of enough key streams (around 60) can be observed, then the WEP key can be recovered through these weak keys using an FMS attack, named after the authors of the paper.

The FMS attack utilizes the fact that, in some cases, knowledge of the IV and the first output byte leaks information about the key bytes. This attack itself is already a problem but it is even made worse because of another implementation error by WEP: the first couple of bytes of encrypted WEP-data in every packet ARE known. The LLC/SNAP header encapsulating some higher layer protocol is always the first 8 bytes in the encrypted packet, namely 0xAA, in other words: we have a part of the original plain text.

How can this be used? Suppose a plain text p_i, where i denotes the number of blocks, which is encrypted with a key stream k_i. This produces the cipher text c_i:

$c_i = k_i \oplus p_i$

An interesting property of all one-time pad ciphers, like RC4, is the following:

$c_i = k_i \oplus p_i \Leftrightarrow k_i = c_i \oplus p_i$

Suppose p_i is the first 8 bytes of the known plain text. If these known bytes (the LLC/SNAP header) are XOR'd with the first 8 bytes of the cipher text, the first bytes of the key stream are reproduced. Which of course is exactly the part needed to start an FSM-attack. This part of the key stream can now be used to recover the rest of the RC4 key. An attacker now has all the ingredients to read any cipher text using this key, since both the IV and the PRNG are always known. Making it possible to create the needed key streams to decrypt any captured packet, or vice versa: encrypt any chosen data and sent it to anyone, pretending to be an authorized user.

Two very popular Linux programs utilize the FMS-attack: Airsnort & WEPCrack.

IV collisions occur If the FMS attack itself is not disastrous enough on its own, a whole other breed of flaws results from the fact that the IV is only 24 bit large. The use of a 24-bit IV is inadequate because the same IV, and therefore the same key stream, must be reused within a relative short period of time.

A 24-bit field can contain 2^{24} or 16 777 216 possible values. A short calculation demonstrates the short life of a 24-bit IV.

Given: a slow access point running at 11 Mbps and constantly transmitting 1.500-byte packets:

- 11 Mbps / (1.500 bytes per packet x 8 bits per byte) = 916.67 packets transmitted each second
- 16.777.216 IVs / 916.67 packets per second = 18.302,41745 seconds

That means that after a little bit more than 5 hours all IVs are used up and collisions will start occuring.

This flaw can be abused in two ways, by passively attacking the network, or actively flooding the network with data and retrieving the key streams because of the collisions.

Passive attack A passive eavesdropper can quietly intercept all wireless traffic, until an IV collision occurs. By XOR'ing two packets that use the same IV, the attacker obtains the XOR of the two plaintext messages. The resulting XOR can be used to retrieve information about the contents of the two messages.

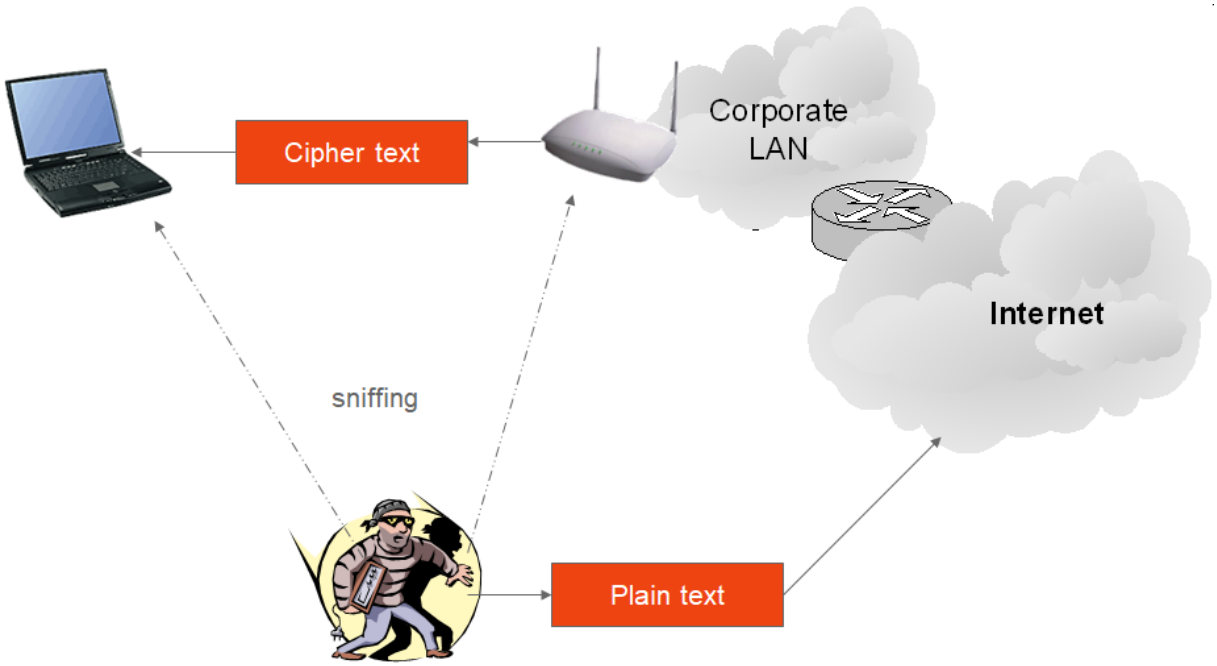
IP traffic is often very predictable and includes a lot of redundancy. This redundancy can be used to eliminate many possibilities for the contents of messages. Further educated guesses (through means

of cryptanalysisy) about the contents of one or both of the messages can be used to statistically reduce the space of possible messages, and in some cases it is possible to determine the exact contents, an example of this was given earlier where the LLC-header was used to launch an FMS attack.

When such statistical analysis is inconclusive based on only two messages, the attacker can look for more collisions of the same IV. With only a small factor in the amount of time necessary, it is possible to recover a modest number of messages encrypted with the same key stream, and the success rate of statistical analysis grows quickly. Once it is possible to recover the entire plaintext for one of the messages, the plaintext for all other messages with the same IV follows directly, since all the pairwise XOR's are known.

Active attack An extension to this attack uses a host somewhere on the Internet to send traffic from the outside to a host on the WLAN installation. The contents of such traffic will be known to the attacker, yielding the known plaintext. When the attacker intercepts the encrypted version of his message sent over 802.11, he will be able to decrypt all packets that use the same initialization vector.

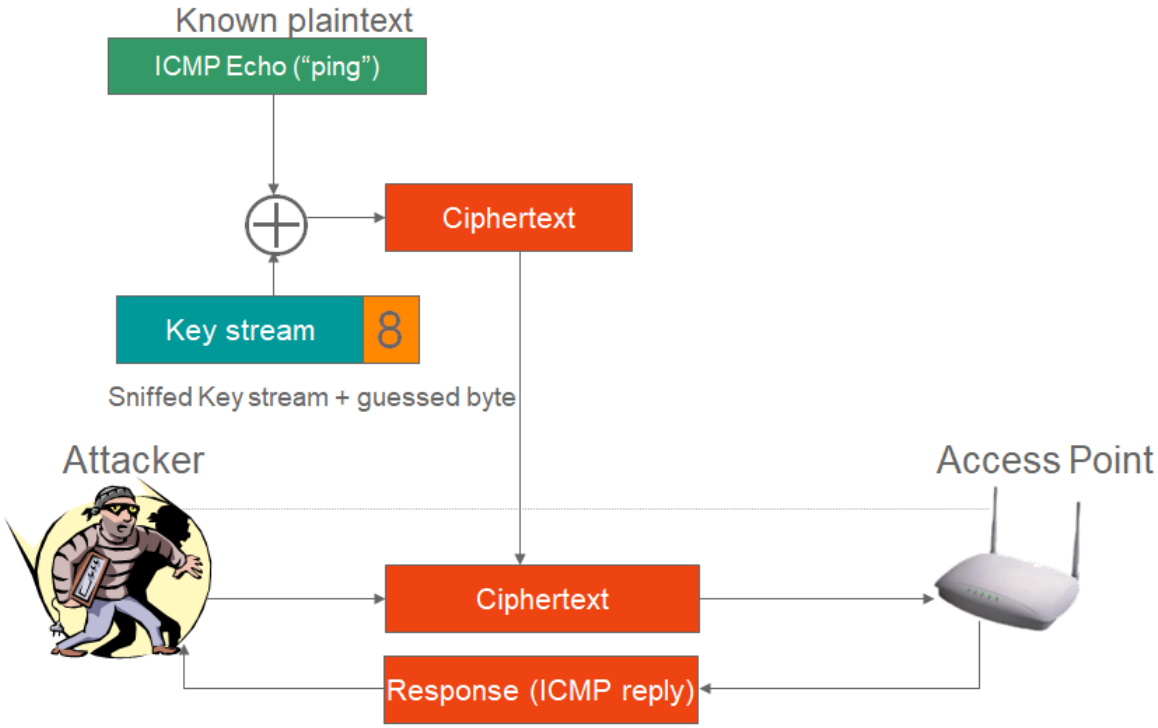
An attacker could use the Internet for this type of attack:



1. A known plain-text message is sent to an observable wireless LAN client (an e-mail message). If the attacker is only capable of utilizing the WLAN he will have to use a bit-flip attack, explained later on.
2. The network attacker will then start sniffing the wireless LAN looking for the predicted cipher-text and eventually find it.
3. The network attacker will find the known frame and derive the key stream using the reverse XOR action: $c_i = k_i \oplus p_i \Leftrightarrow k_i = c_i \oplus p_i$. Where k_i is the desired key stream.

Growing key streams When the attacker has one key stream, the story does not end here. With this key stream, the attack can now create any key stream, of any desired length due to the fact that there is no replay protection. WEP doesn't check if a frame sent is authentic or not, if a frame is encrypted with the right key, WEP considers the user to be valid. Someone can capture a packet and resend it at any given time; it will not be checked and thus be used as if it were a valid packet. An attacker can thus use the WLAN as was it is own private laboratory where he can test as much as desired.

And so the attacker can grow his own key stream of any given length :



1. The network attacker can build a frame one byte larger than the known key stream size; an Internet Control Message Protocol (ICMP) echo frame is ideal because the access point solicits a known response.
2. The network attacker then augments the key stream by one byte.
3. The additional byte is guessed because only 256 possible values are possible and he can 'guess' as many times as he wants.
4. When the network attacker guesses the correct value, the expected response is received: in this example, the ICMP echo reply message. Otherwise he won't get anything since the packet was encrypted with an invalid key stream.
5. The process is repeated until the desired key stream length is obtained.

IV Selection The fourth problem with the IV is how it needs to be selected. The 802.11 standard specifies no rules for IV selection, instead it merely recommends updating "frequently", a pretty undefined term. Each vendor implemented his or her own IV selection strategy, some being smarter then others:

- Fixed IV: Some implementations operate with a fixed IV, employing the same RC4 key to encrypt every packet. Making it necessary that the RC4 key needs to change after each packet, since a collision occurs afterwards with the second packet!
- Random IV: Other vendors selected the IV at random, seemingly the best solution but actually not much better then the former solution, all due to the infamous **birthday paradox**. Because this paradox it only takes 4823 packets to have a 50% chance of collision. And so a key change needs to occur about every three seconds.
- Incremental IV: A third option is to have a circular counter, incrementing the IV after each transmission, starting always from zero upon boot. This strategy guarantees a collision after two different stations transmit a single packet, since it is common to use only default keys (i.e. the same key on every device).

It is clear that 16.777.216 possible values are not enough in a high data-rate environment, such as in a WLAN, and so the IV is a severe weakness.

No matter what IV sequencing formula is used, a strong key management system is required, which is not available. This key management could solve the IV problem by including a system that would change the keys before all the IV possibilities are exhausted, thus creating new key streams.

Problem 3: CRC

WEP uses an integrity checksum field to prevent a packet from being modified during transmission, using a CRC-32 checksum. This wasn't the best choice: CRCs in general are designed to detect random errors in a message (such as sudden line interference, noise, etc), not to detect planned forgeries.

This, and the fact that a stream cipher is used to encrypt the payload, gives raise to 1 more group of vulnerabilities in WEP.

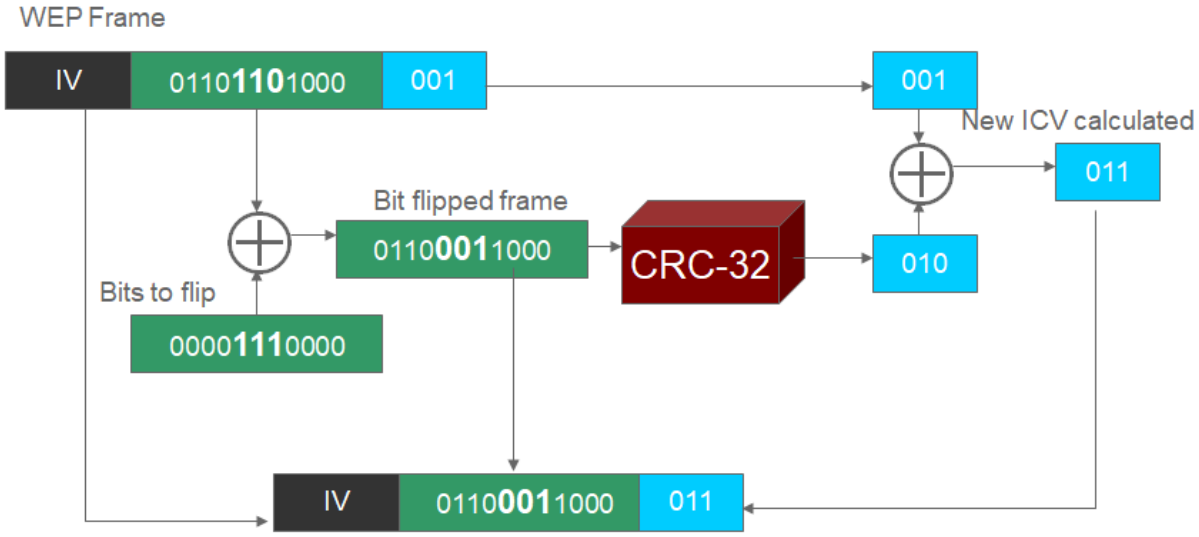
The WEP checksum is a linear function of the message, as is with all CRCs. Meaning that the CRC of one message XOR'd with the CRC of another message, is the same as the CRC of two XOR'd messages:

$$CRC(message_1) \oplus CRC(message_2) = CRC(message_1 \oplus message_2)$$

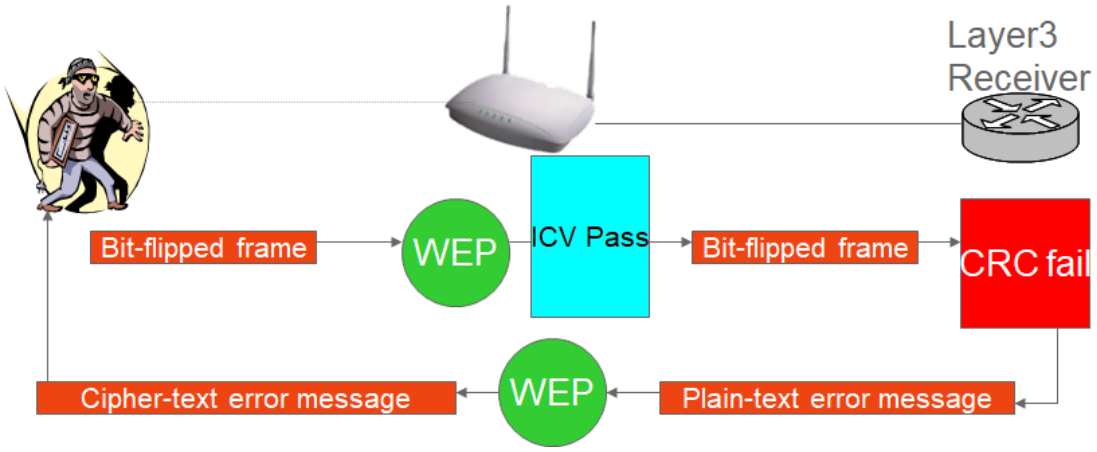
As a consequence it becomes possible to make a controlled forgery without disrupting the checksum. To do this, an attacker employs a 'bit-flip attack'.

Bit-flip attack A bit-flip attacks enables an attacker to inject his own messages without the receiver being able to detect that it is not the original message.

Firstly the attacker captures a valid WEP frame (without needing to known the plaintext context). He then XOR's this frame with a self created stream of bits, where the 1 bits cause the bit to flip (0 to 1, 1 to 0), the so-called bitmask. Usually this stream of bits is randomly. As will be shown, the attacker just needs a valid frame, whatever the data it caries. Lastly the attacker, to have a valid ICV, XOR's both ICVs of the new and original frame. Since XOR'ing is commute (the order of the operations is not relevant) a new valid hash is created.



The attacker can now send a forged frame that is considered genuine by the receiver. The receiver (AP) dutifully decapsulates the bit flipped data and the LLC discovers that the data is 'gibberish'. Yet, since the ICV was valid, the receiver simply thinks some higher layer CRC error has occurred and thus sends an encrypted error-message to the attacker.



It is this error-message that the attacker knows he will receive, so encrypted or not, the attacker can

now recreate the key stream by XOR'ing the encrypted error-message with the original error-message as shown as explained before: $c = k \oplus p \Leftrightarrow k = c \oplus p$

Once this key stream k is derived, the attacker can have a key stream of any size by 'growing' one, as described earlier. By doing this, the hacker can create a dictionary of keystreams of all sizes. From then on, the attacker can transmt any message of any size, since he has valid keystreams

Note that the attacker in this scenario doesn't need to have the WEP-key. He simply uses the keystreams to mimick being a valid user.

Problem 4: Keying

A problem, where most symmetric systems suffer from, is the key distribution. The default keys need to be distributed to all the stations that want to participate in the service set secured by WEP. There is, however, no key distribution specified in the 802.11 standard. And so, vendors haven't done anything: most devices just have you type the keys manually in to the device drivers or APs. (Some provide a way of distributing the keys on a small disk with an executable).

It is clear that this manual entry is entirely dependent of the users and system manager, not of the protocols, creating a very insecure key environment:

Keys cannot be considered secret: all keys need to be manually entered, and any local user, with a minor knowledge of his system, can extract the keys (e.g. in Windows OS through the device manager).

Whenever someone, a staff member for example, leaves the organization, all keys should be changed. Knowledge of WEP keys allows a user to setup a station and passively monitor and decrypt traffic using the secret key. WEP thus cannot protect against authorized insiders who also have the key. Organizations with a large number of authorized users must publish the key to this group when needed, which, of course, prevents the keys from being secret.

Phase 2: WPA 1

Phase 3: WPA 2

Phase 4: WPA 3 ("Wifi 6")

Een oprechte dank aan Stefaan Somerling (RealDolmen) voor de feedback op het hoofdstuk omtrent GDPR. Geef gerust een sein als u, conform de GDPR wetgeving, liever uw naam niet in dit document ziet staan ;)